

HAND-OFF: fiveup / ProofSignals — Discord-backed matching, points and Skool tie-back

You are the build agent for this refactor. Read this whole file before touching code. The full plan is in `docs/refactor/REFACTOR-PLAN.md` (Robbie will drop the PDF/markdown there; if it is missing, ask for it before starting). This file is the operating contract: locked decisions, conventions, the first two phases in detail, and the questions still open.

0. Working rules (non-negotiable)

- 1. Branch per phase off `main`.** Name: `phase-a-shelve-and-switch`, `phase-b-points`, `phase-c-discord-commands`, `phase-d-graph-squads`, `phase-e-redemptions`, `phase-f-review-integrity`. Open a PR per phase. Robbie reviews and merges. If the previous phase's PR is not merged yet when you start the next, branch off that unmerged branch (stacked PRs) and say so in the new PR description.
- 2. Never delete member data or drop a table.** Archive means: move code to `src/archive/`, keep the route, keep the RPC, gate it. Every migration must be safely re-runnable (`create ... if not exists`, `create or replace`, `idempotent seeds`), matching the style of every existing file in `supabase/migrations/`.
- 3. Follow the repo's existing patterns exactly.**
 - Migrations: `supabase/migrations/YYYYMMDDHHMMSS_<phase>_<topic>.sql`, header comment explaining what and why (see `20260910140000_phase15_match_preferences.sql`).
 - All writes go through `security definer` RPCs with `set search_path = public`, `auth.uid()` checks, `assert_active_caller()`, and admin override via `is_admin()`.
 - RLS enabled on every new table. Direct table writes revoked for `authenticated`.
 - `set_updated_at` trigger on every table with `updated_at`.
 - Data access from the app goes in `src/lib/fivestarz/data.js` (one exported function per RPC), never inline Supabase calls in components.
 - Server-only helpers live in `src/lib/fivestarz/` and are imported by

`src/app/**/actions.js` route handlers.

- o Every phase ships a `verify-phaseNN.mjs` at the repo root in the style of `verify-phase16.mjs` (no DB, no dev server; asserts composition, wiring and source-level invariants) plus a `docs/fiveup-continuation-phaseNN.md` written like the existing phase docs (what changed, migration, verify, out of scope).

4. **Do not touch** `proof_lab_*` table names, the moderation subsystem, or the public visibility model except where a phase explicitly says so.
5. **Ask before inventing.** Anything not covered here or in the plan: add it to the PR description under "Decisions I made" so Robbie can veto at review.

1. Locked decisions (from Robbie, Sep 24, 2026)

#	Topic	Decision
1	Marketplace name	Rename the "Proof Lab" marketplace to Proof Market . Routes <code>/proof-lab</code> → <code>/proof-market</code> (keep a redirect from the old path). Rename components, nav labels, page titles, OG copy, the dashboard tab, <code>notify-seller</code> email copy. Tables, enums and RPC names stay as-is (<code>proof_lab_*</code>).
2	Marketplace during refactor	Stays live . Browsing and buying open to all ProofSignals members; creating listings only for paying ProofSignals plans (the existing <code>proof_lab_listings_enabled</code> gate already does this: <code>sprout=false</code> , <code>bloom/flourish=true</code>). Robbie (admin) can create listings for free members until then.
3	Plans	ProofSignals plans (<code>sprout</code> / <code>bloom</code> / <code>flourish</code>) are separate from Skool and do not connect. One exception: flourish includes the paid Skool group (not the other way round). Skool has only a Free Group and a Paid Group — no special plan names in code. Implementation: a <code>flourish</code> activation creates a queue item "invite to paid Skool" for an admin (manual on Skool Hobby; Zapier <i>Invite Member</i> later on Skool Pro).
4	Point values	Keep it simple: every accomplishment = 1 point, every assist = 2 points . Both require a before/after proof upload (screenshot or printed audit) where possible. Proof is a required field on the claim form; admin may approve without it, with a note. No breadth weighting for now.

5	Point value in dollars	1 point = \$5. Redemption cost = <code>ceil(usd_price / 5)</code> . Store <code>point_usd_value = 5</code> in <code>plan_feature_gates</code> config (feature key <code>points_economy</code>) so it can change without a deploy. Examples: \$29 Skilllet → 6 points; a Pro month priced at \$X → <code>ceil(X/5)</code> .
6	Skool plan	Hobby. Zapier integration deferred. Note for later: unlocking a Skool Course can be the delivery mechanism for an app or Skilllet — the course page holds a link to a Lemon Squeezy checkout with a pre-applied 100%/50% code. Design the redemption fulfilment so a <code>fulfillment_kind = 'skool_course_unlock'</code> can be added later without a schema change.
7	Queue approvers	Any admin or moderator. Points-queue RPCs check <code>is_admin()</code> or <code>is_moderator()</code> , exactly like <code>list_moderation_queue / resolve_flag</code> .
8	Discord identity	Open — see Section 5, question 1. Build Phase A and B first; they do not need it.
9	Workflow	Branch → PR → Robbie merges → next branch off <code>main</code> (or off the unmerged branch). See Section 0.
10	Email sender	Move from <code>noreply@bendersaas.ai</code> to indieops.co . See Section 4 for the exact steps and files.

2. Phase A — Shelf and switch (branch `phase-a-shelve-and-switch` , ~1 week)

Goal: the web matching surface is archived behind one gate; the marketplace is renamed; flipping the gate to `web` restores everything. Merge only after the restore is proven on a preview deploy.

A1. Tag and branch

- `git tag pre-discord-refactor` on `main` and push the tag.
- Create the branch.

A2. The gate

- Migration `..._phaseA_match_surface_gate.sql` : seed `plan_feature_gates` rows for feature key `match_surface` on all three plans with `config = '{"surface":"discord"}'` .

Add RPC `public.match_surface()` (stable, security definer) returning the caller's plan's surface, defaulting to `'web'` when unset, so a missing row can never lock members out.

- `create_match` and `eligible_match_candidates` : add a first-line guard — proceed only if `match_surface() = 'web'` **or** the caller is the service role (`auth.role() = 'service_role'`) **or** `is_admin()` . Raise `'matching is handled in Discord'` otherwise. Same function bodies otherwise; `create` or `replace` in place.
- Add a `discord` value to the informal set of `matches.source` values (it is free text — document the convention in the migration header, no enum change).

A3. Archive the web matching UI

- Create `src/archive/web-matching/` and move there, unchanged: `BrowsePage.jsx` , `MatchPreferencesPage.jsx` , and the match request/accept/feedback-modal section of `DashboardPage.jsx` extracted into `MatchActions.jsx` (leave the dashboard's read-only match history working; only the *initiating* actions move).
- Routes stay: `src/app/browse/page.jsx` and `src/app/account/preferences/page.jsx` read the gate server-side and either render the archived component (`web`) or redirect to `process.env.NEXT_PUBLIC_DISCORD_INVITE_URL` (`discord`). No 404s.
- In `data.js` , prefix `getBrowseQuota` , `getEligibleCandidates` , `requestMatch` with `// ARCHIVED: web matching surface — see docs/archive/web-matching.md` . Do not delete.
- `SiteNav.jsx` : hide "Browse Members" when the surface is `discord` .
- Write `docs/archive/web-matching.md` : what was archived, why, and the restore procedure (update one config row to `web` ; nothing else). **Test the restore on a Vercel preview and record the result in the PR.**

A4. Rename marketplace to Proof Market

- Route folder `src/app/proof-lab/` → `src/app/proof-market/` ; add `src/app/proof-lab/page.jsx` that permanently redirects to `/proof-market` .
- Components: `ProofLabPage.jsx` → `ProofMarketPage.jsx` , `ProofLabTeaser.jsx` → `ProofMarketTeaser.jsx` ; update all user-facing strings ("The Proof Lab" → "Proof Market"), nav link, dashboard tab label, sitemap, OG titles, and the `notify-seller` email copy.
- Keep the API route path `src/app/api/proof-lab/notify-seller/route.js` (internal; renaming is churn without benefit) — or move it and update the one caller; your call, note it in PR.

- `verify-phaseA.mjs` : asserts the redirect exists, the archive folder is imported only by the gated routes, no remaining “Proof Lab” strings in `src/components` and `src/app` except the redirect, and the migration guard text is present in the two RPCs.

3. Phase B — Points without Discord (branch `phase-b-points` , ~2 weeks)

Goal: a member can claim an accomplishment or nominate an assist with proof, an admin or moderator approves it from `/admin` , the ledger updates, and the numbers show on the public profile. Nothing here needs Discord.

B1. Migration `..._phaseB_points_ledger.sql`

Tables (RLS on, writes via RPC only, `set_updated_at where updated_at exists`):

- `interactions` — `id` `uuid` `pk` , `kind` `text` `check in ('assist','collaboration','squad','internal_review','external_review_event','match')` , `member_a` `uuid` , `member_b` `uuid` , `context_type` `text` , `context_id` `uuid` , `evidence_url` `text` , `occurred_at` `timestampz` , `source` `text` `check in ('discord','web','admin')` , `created_at` . **Trigger after insert: call `refresh_member_match_edge_for_pair(member_a, member_b)` and `bump_member_graph_version()` so every interaction feeds separation degree.** Read the existing edge refresh function first; if it only understands `matches` rows, add a minimal variant that upserts the edge from an interaction rather than rewriting it.
- `accomplishments` — **catalog:** `id` , `slug` `unique` , `title` , `description` , `points` `int` `default 1` , `requires_proof` `bool` `default true` , `active` `bool` `default true` , `sort_order` . **Seed:** `ship-app` , `ship-skilllet` , `first-paying-client` , `twilio-10dlc-approved` , plus `custom` (admin-only, needs note). Admin-editable via RPC, not hard-coded in JS.
- `accomplishment_claims` — `id` , `user_id` , `accomplishment_id` , `note` , `proof_before_path` , `proof_after_path` , `evidence_url` , `status` `text` `check in ('pending','approved','rejected')` , `reviewed_by` , `reviewed_at` , `review_note` , `created_at` , `updated_at` .
- `assists` — `id` , `helper_user_id` , `helped_user_id` (must equal `auth.uid()` on create; `helper` $\neq$ `helped`), `interaction_id` , `what` `text` , `proof_before_path` , `proof_after_path` , `evidence_url` , `status` , `reviewed_by` , `reviewed_at` , `review_note` , `timestamps` .

- **points_ledger** — **append-only**: `id`, `user_id`, `delta` `int`, `kind` `text` check in ('accomplishment','assist','admin_adjust','redemption','reversal'), `source_table` `text`, `source_id` `uuid`, `note`, `created_by`, `created_at`. **No update/delete policies for anyone; corrections are new rows.**
- **redemptions** — `id`, `user_id`, `kind` `text` (`skilllet`, `app_month`, `pro_month`, `pro_year`, **extensible**), `item_ref` `text` (e.g. `Skilllet slug` or `app name`), `points_cost` `int`, `usd_value` `numeric`, `status` `text` check in ('requested','approved','fulfilled','refunded','rejected'), `fulfillment_kind` `text`, `fulfillment_note` `text`, `reviewed_by`, **timestamps**. (Table now; spend RPC ships in Phase E.)
- **Columns on** `user_profiles`: `points_balance_cached` `int` default 0, `lifetime_points_cached` `int` default 0, `assists_given_cached` `int` default 0, `accomplishments_cached` `int` default 0. **Trigger on** `points_ledger` **insert** **refreshes** them (pattern: `refresh_profile_feedback_rating`). **Lifetime** = sum of positive approved deltas; **balance** = sum of all deltas.
- **Storage: private bucket** `proof-uploads`, **objects at** `<user-id>/<claim-or-assist-id>/<before|after>.<ext>`, **owner-write, admin/moderator-read, mirroring** `20260308130000_asset_screenshots_storage_bucket.sql`.
- **Seed** `plan_feature_gates` **feature** `points_economy` **on all plans with** `config = '{"point_usd_value":5,"accomplishment_points":1,"assist_points":2}'`.

RPCs (all security definer):

- `claim_accomplishment(p_slug, p_note, p_evidence_url, p_before_path, p_after_path)`
- `nominate_assist(p_helper_user_id, p_what, p_evidence_url, p_before_path, p_after_path)` — **also inserts the** `interactions(kind='assist', source='web')` **row**
- `list_points_queue(p_status default 'pending')` — **admin or moderator; unions** pending claims and assists with member display names and proof paths
- `resolve_points_item(p_item_type, p_item_id, p_action, p_note)` — **admin or moderator; approves/rejects; on approve inserts the ledger row (** `delta` **from** `points_economy` **config, not from the client) and returns the new balance**
- `admin_adjust_points(p_user_id, p_delta, p_note)` — **admin only**
- `get_public_points(p_username)` — **anon-readable: lifetime points, accomplishments count, assists given, gated by the profile's existing** `show_stats` **publishing setting**

- `upsert_accomplishment(...)` — admin catalog editing

B2. App

- `data.js` : one function per RPC above, plus `uploadProofFile(supabase, kind, id, file, which)` modelled on `uploadAssetScreenshot`.
- Dashboard: new tab **Points** — balance, lifetime, pending items, “Claim an accomplishment” form (catalog dropdown, note, evidence URL, before/after uploads), “Thank someone for an assist” form (member picker by public username, what they helped with, uploads).
- `/admin` : a **Points queue** section beside the moderation queue, same table layout, approve/reject with note, proof thumbnails via signed URLs. Reuse the moderation outcome-email pattern: add `composePointsEmail({action, itemType})` in a new `src/lib/fivestarz/points-email.js`, best-effort, never blocks the action.
- `/u/[username]` : a **Proof** block showing lifetime points, accomplishments (titles + dates), assists given; hidden when `show_stats` is off. Extend the profile OG image to include lifetime points and star average so the link preview is the badge.
- Import: a one-off script `scripts/import-assist-sheet.mjs` that reads a CSV `( date, helper_username, helped_username, what, link )` and creates pre-approved assists + ledger rows as admin. Robbie’s hand-kept Skool assist sheet is the input.
- `verify-phaseB.mjs` : ledger is append-only (no update/delete policy text), delta comes from config not client, helper ≠ helped enforced, queue RPCs gate on admin-or-moderator, public RPC respects `show_stats`.

Phases C–F

Follow `docs/refactor/REFACTOR-PLAN.md` sections “Discord bot”, “Data model changes”, “Points economy in code”, “Skool tie-back”, and “Phased build plan”. Do not start Phase C until Section 5, question 1 is answered.

4. Email sender move to indieops.co (do in Phase A)

Robbie’s domain and email are hosted at SiteGround; DNS is managed there.

1. In Resend → Domains → Add Domain: use a **subdomain**, `notify.indieops.co` (protects the root domain’s reputation; Skillet and thank-you mail can use others later).
2. Resend shows three records. Add them in SiteGround → Site Tools → Domain → DNS

Zone Editor:

- DKIM: TXT at `resend._domainkey.notify` (value from Resend)
- SPF: TXT at `notify` → `v=spf1 include:amazonses.com ~all` (exact value from Resend)
- MX at `notify` → the Resend/SES feedback host Resend shows (return-path)
- Add DMARC once: TXT at `_dmarc.notify` → `v=DMARC1; p=none; rua=mailto:dmarc@indieops.co`

3. Click Verify in Resend; wait for green.

4. Code: replace the hard-coded sender in **both** places with an env var:

- `src/app/api/proof-lab/notify-seller/route.js` line ~52
- `src/lib/fivestarz/moderation-email.js` line ~111 → `from:`
`process.env.RESEND_FROM ?? "ProofSignals <noreply@notify.indieops.co>"`, and change the display name from "FiveStarz" to "ProofSignals".

5. Add `RESEND_FROM` to `.env.example` and to Vercel production env. `RESEND_API_KEY` is already production-only; keep it that way.

6. Send one test of each email type from production and paste the headers (DKIM pass) in the PR.

5. Open questions — ask Robbie before the relevant phase

Q1. Discord identity linking (blocks Phase C) — the options, explained

The bot must know which ProofSignals account a Discord user is. There are three ways; pick one for Robbie to confirm.

Option 1 — Discord as a login provider + identity linking (recommended). Supabase Auth supports "Sign in with Discord". You create one Discord Application in the Discord Developer Portal (the same application the bot uses), copy its Client ID and Secret into Supabase → Authentication → Providers → Discord, and add Supabase's callback URL to the app's OAuth2 redirects. Then, in Supabase → Authentication → Settings, enable **manual linking** so an existing member can attach a second login. On `/account`, a **Link Discord** button calls `supabase.auth.linkIdentity({ provider: 'discord' })`; Discord's own OAuth screen confirms who they are and Supabase stores the Discord user id on the account. The handler copies it into `member_identities`. New members can also *sign up* with Discord, in

which case the link is automatic. Pros: Discord itself vouches for the id, no codes to type, no support tickets, scales to any number of members, standard and boring. Cons: ~30 minutes of one-time setup; members must click one button on the site.

Option 2 — `/link` code flow only. The member types `/link` in Discord; the bot (whose requests Discord signs, so the Discord id is trustworthy) generates a 6-character code valid for 10 minutes and stores it against the Discord id; the member pastes the code into `/account`. Pros: no OAuth configuration. Cons: manual, expiring codes, typos, "it didn't work" DMs, and a second table to maintain. Fine for 100 members; a support burden at 10,000.

Option 3 — Both. Option 1 as the primary path, Option 2 as a fallback for members who struggle with OAuth on mobile.

Recommendation: **Option 1 now**, add Option 2 only if support requests demand it. This is also the most reliable at scale because the identity is established by Discord's OAuth, not by a human copying a code.

Q2. Skool identity

With no Zapier (Hobby plan), the only reliable Skool identity link is the member pasting their Skool profile URL on `/account` (stored in `member_identities`, `provider skool`, `unverified` until an admin eyeballs it). Confirm this is acceptable for now, and whether the public profile should show the Skool link.

Q3. Proof requirement strictness

"Require proof where possible": should the claim form **block** submission without both before/after uploads, or allow submission with a warning and let the approver decide? (Plan assumes: required in the form, approver can override with a note.)

Q4. Flourish → paid Skool

On Hobby there is no automation. Confirm the flow: `flourish` activation creates an admin queue item "Invite <email> to paid Skool group"; admin invites manually; item marked done.

Q5. Accomplishment catalog additions

Beyond ship-app, ship-skilllet, first-paying-client, twilio-10dlc-approved: any others for launch? (E.g., "published a public ProofSignals profile", "completed first Squad mission".)

Q6. Redemption catalog for Phase E

Which items are spendable at launch and their USD prices (Skilllet \$29 → 6 points is assumed): app month prices for CommentCraft / ReviewReels / ReviewCute / GBP-Magnet /

PropStar, Pro month, Pro year.

6. First commands for the agent

```
git checkout main && git pull
git tag pre-discord-refactor && git push origin pre-discord-refactor
git checkout -b phase-a-shelve-and-switch
cat docs/refactor/REFACTOR-PLAN.md
```

Then execute Section 2 top to bottom, run `node verify-phaseA.mjs` , write `docs/fiveup-continuation-phaseA.md` , and open the PR with: what changed, migration summary, the restore test result, decisions you made, and anything from Section 5 you needed and did not have.